

Project Schedule Analysis

1. Task Dependencies

During Sprint 2, the team worked on several functional components of the system, including the therapist dashboard, appointment booking features, monitoring modules, and the admin portal. Many of these tasks had logical dependencies that influenced the order in which they could be implemented.

For example, the booking screen implementation needed to be completed before users could view upcoming and past appointments. Similarly, the admin dashboard foundation had to be implemented before administrative features such as user list management and user detail management could be developed.

The main dependency relationships are summarized below:

Start

- Booking Screen Implementation (SCRUM-37)
- View Upcoming and Past Appointments (SCRUM-13)
- Therapist Dashboard Showing Performance and Patient Data (SCRUM-29)

Start

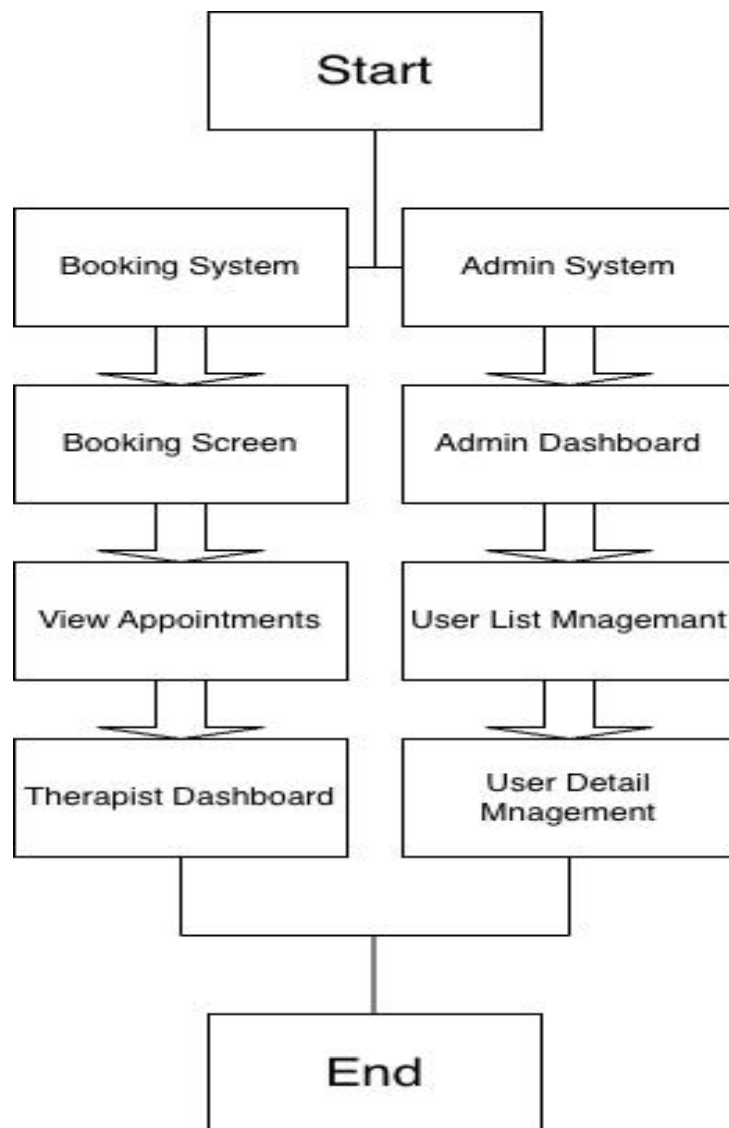
- Build Admin Dashboard Foundation (SCRUM-44)
- Implement Admin Users List Management (SCRUM-45)
- Implement Admin User Details and Status Management (SCRUM-46)

In addition, system monitoring features such as Daily Monitoring (Health Metrics / Medications / Medical Records) support the overall platform functionality and integrate with the broader educational and patient-care modules.

These dependencies ensured that foundational infrastructure was implemented before higher-level features relying on that infrastructure.

2. Network Diagram

The following simplified network diagram illustrates the logical sequence of development tasks during the sprint.



This diagram shows how foundational components such as booking functionality and the admin dashboard enable more advanced features within the platform.

3. Critical Path

The critical path represents the sequence of tasks that directly determine the minimum time required to complete the sprint objectives.

In this sprint, the most important development chain was related to the core service flow of booking and appointment management. The critical path can be summarized as:

Booking Screen Implementation
→ View Upcoming and Past Appointments
→ Therapist Dashboard Display

These features represent the primary interaction workflow between patients and therapists. If any task in this sequence were delayed, the entire functionality of appointment management and therapist interaction would be affected.

Similarly, the admin management workflow forms another important development chain:

Admin Dashboard Foundation

→ Admin Users List Management

→ Admin User Details and Status Management

Although both chains are important, the booking and appointment workflow is considered the primary critical path because it directly supports the platform's core healthcare service functionality.

4. Risk Areas

Several potential risk areas were identified during Sprint 2.

First, feature dependencies between frontend and backend components created integration risks. For example, the therapist dashboard required data from the appointment service and patient monitoring modules. If these backend services were not fully implemented or stable, it could delay dashboard development.

Second, changes in requirements after discussions with the project founder introduced some uncertainty. Feature expectations sometimes evolved during the sprint, requiring the team to refine user stories or adjust implementation details.

Third, integration testing posed a risk due to the complexity of connecting multiple system components, including booking, monitoring, and administrative management. Bugs discovered during integration could potentially delay feature completion.

To mitigate these risks, the team maintained regular meetings, used Jira to track progress and dependencies, and clarified requirements with stakeholders whenever necessary.

5. Lessons Learned

From the sprint execution and schedule analysis, several lessons were identified.

First, defining task dependencies earlier in the sprint planning phase would help improve task coordination and reduce potential blocking issues between development tasks.

Second, assigning story points at the beginning of the sprint would allow more accurate tracking of development velocity and progress through the burndown chart.

Finally, maintaining clear documentation and consistent Jira management practices helps the team better understand task relationships and ensures smoother sprint execution.